

# Project ESCAPE: Structural Discovery via Entropy in the Byte Latent Transformer Across Code and Natural Language

Priyanka Agarwal  
2024101016

Shashwat Mukadam  
2024102008

Divyansh Atri  
2024113001

## Abstract

The Byte Latent Transformer (BLT) replaces fixed subword vocabularies with a dynamic, per-input decision: a small byte-level patcher reads raw bytes and starts a new computational patch whenever its own next-byte entropy crosses a threshold. Because this decision is made live, with no parser and no syntax supervision anywhere in training, it raises a question the original BLT work never tested: do these entropy-triggered boundaries land on anything linguistically or grammatically meaningful? We test this directly, calling it the *Grammatical Branching Hypothesis*, on Python, C++, and English, scoring BLT’s boundaries against tree-sitter ASTs (code) and benepar constituency parses (prose), against a density-matched null model and against structure-blind line/word baselines. BLT’s boundaries align with constituent starts far more than chance (F1 margins of +0.12 to +0.35 over a 10,000-permutation null), but are consistently beaten by simple baselines, splitting at every newline or every word start, in all three languages. Two further predictions reverse outright: alignment is *stronger* in English prose than in code, and *stronger* at grammatically forced openers (*if*, *for*, *while*, *def*) than at open-ended constructs, both opposite to what the hypothesis’s own mechanism predicts. We argue these are not three independent failures but one finding seen three ways. BLT’s entropy signal is substantially explained by surface segmentation (where lines and words begin), rather than by grammatical branching specifically. A fourth prediction, robustness to code-level noise, holds as expected. We discuss the implication for reading byte-level compute allocation as evidence of linguistic structure, and lay out the remaining work needed before the hypothesis can be considered fully evaluated.

## 1 Introduction

### 1.1 Motivation

Tokenization has been the fixed front end of language modeling for a decade. A byte-pair-encoding (BPE) vocabulary (Sennrich et al., 2016) is learned once, before a model ever sees its training data, and after that every input is segmented the same way regardless of content: a tokenizer spends the same number of tokens on repetitive boilerplate as on a single dense, information-bearing line. The Byte Latent Transformer (Pagnoni et al., 2025) removes this fixed vocabulary entirely. A small byte-level language model, the *patcher*, reads raw bytes and, at each position, measures its own uncertainty about the next byte via Shannon entropy. When that entropy crosses a threshold  $\tau$ , BLT starts a new patch, so a larger transformer downstream spends more compute exactly where the patcher found the next byte hard to predict, and less where it did not.

Adaptive and conditional computation has a long history in sequence modeling, but BLT is distinctive in making the allocation decision from raw bytes, with no intermediate symbolic representation and no task-specific signal guiding where to spend compute: any structure its boundaries track has to come from the statistics of the bytes themselves. This makes a patch boundary something genuinely new: not a preprocessing decision fixed before training, but a claim the model makes, live, about its own uncertainty, in principle checkable against an independent ground truth in a way a BPE merge table never allows. The BLT authors demonstrate the approach’s efficiency and downstream accuracy at scale but never check whether the induced boundaries correspond to anything externally verifiable. That is the gap this project addresses, and code is a natural place to look: it has a machine-checkable ground truth in its abstract

syntax tree (AST), with grammar choice points that free-form prose does not share in the same way.

## 1.2 The Grammatical Branching Hypothesis

We state this as a narrow, falsifiable claim, not an aspiration: **BLT’s entropy-triggered patch boundaries concentrate at the start of syntactic constituents, more so in code than in prose, and more so at constituent starts than at their ends.** We give an explicit mechanism for why this should hold (§3), a null model and baseline suite that must be beaten before any alignment number means anything (§4), and, critically, an explicit account of where the mechanism itself predicts it should *fail*. A hypothesis that cannot fail in a specified way is not being tested; two of our four secondary predictions do in fact fail at exactly the points the mechanism names.

## 1.3 Contributions

1. A stated, falsifiable mechanism for why entropy-triggered boundaries should track constituent starts, including an explicit account of where it must fail (§3).
2. A null model (density-matched permutation) and a structure-blind baseline suite (newline, indentation-change, word-start) that isolate grammar-specific signal from boundary density and from trivial surface segmentation (§4).
3. A built-in negative control: English prose, scored with the identical protocol as code, so a failure to separate code from prose is itself evidence about what the mechanism measures (§4, §6).
4. A robustness check under code-level noise, and an explicit, pre-registered account of what an entropy-only mechanism cannot see, namely semantic risk in memory-unsafe C++, left for continued work (§7).

## 2 Related Work

Interpretability work on tokenization has focused almost entirely on static vocabularies. Byte-pair encoding (Sennrich et al., 2016) fixes a merge table before training and has well-documented failure modes, including poor morphological alignment and unstable segmentation of rare or multilingual tokens, but it is a fixed, one-time decision, not something that varies per input. Tokenization-free

architectures remove the vocabulary without removing the uniformity: CANINE (Clark et al., 2022) and ByT5 (Xue et al., 2022) process every byte or character with equal compute, with no mechanism for deciding that one position is more structurally interesting than another. BLT is the first architecture at this scale where segmentation is both byte-level and dynamic, decided per input rather than fixed in advance.

A second, separate line of work has asked whether learned representations encode syntax at all. CodeBERT (Feng et al., 2020) and structural probes on contextual representations (Hewitt and Manning, 2019) show that they can, but only because the model was either trained with an objective entangled with structure, or specifically probed for it with a supervised classifier trained on real parse trees. In both cases the demonstration presupposes access to syntactic supervision somewhere in the pipeline. What has not been asked is whether an *unsupervised, compression-only* signal, with no parser or syntax label anywhere in training, recovers constituent structure as an emergent side effect of simply trying to predict the next byte. That is the specific gap BLT’s dynamic patching opens up, and the gap this project tests directly.

This question also sits inside a broader interpretability concern: as architectures increasingly decide for themselves where computation goes, it becomes harder to tell whether an observed allocation pattern reflects learned structure or is simply an artifact of the allocation mechanism’s own statistics (here, the threshold  $\tau$  and the patcher’s training distribution). A null model that holds those statistics fixed, as in §4, is the minimum needed to separate the two explanations for *any* dynamic-computation architecture, not just BLT.

Using local predictability to decide where a unit begins also has precedent outside neural models: classic unsupervised word-segmentation work infers boundaries from statistical cues such as prediction strength and transition probabilities (Brent, 1999; Goldwater et al., 2009), with no notion of grammar at all; BLT moves the same intuition inside a byte-level neural model, and this project checks the inferred boundaries against the AST the way that literature checked them against the lexicon.

Our evaluation draws on established resources rather than new ones: CodeSearchNet (Husain et al., 2019) and The Stack (Kocetkov et al., 2022) for code, WikiText-103 (Merity et al., 2017) for

prose, tree-sitter for code ASTs, and benepar (Kiptsev and Klein, 2018) for prose constituency parses. We chose benepar because it performs constituency parsing directly, giving a label set that can be organized to mirror the code node-type taxonomy for a like-for-like code/prose contrast.

### 3 The Mechanism

BLT’s patcher is a short-context (effectively 512-byte) byte-level language model; a boundary fires whenever next-byte entropy

$$H_i = - \sum_{v \in \mathcal{V}} p_\theta(v \mid x_{<i}) \log p_\theta(v \mid x_{<i}) \quad (1)$$

crosses  $\tau$ . A boundary therefore marks exactly one thing: a position where the preceding bytes stop determining what comes next. Everything that follows is a consequence of that single fact, and no more.

**Entropy bounds branching; it does not track it.** It is tempting to equate entropy with the local grammatical branching factor  $B$ , the number of valid continuations at a point. It does not measure that directly: entropy depends on the *shape* of the probability distribution over continuations, not merely its support size. Ten valid continuations with one carrying 99% of the probability mass give low entropy; four equiprobable continuations cross most reasonable thresholds. What is guaranteed is only the one-directional bound  $H \leq \log B$ . A boundary is therefore solid evidence that several continuations were live, but a position with many legal continuations is not guaranteed to produce a boundary at all.

**The predicted asymmetry.** Once a grammatical production is committed, partway through `if (x > 0) {`, admissible continuations are sharply constrained: closing delimiters (`;`, `)`, `}`) are among the most predictable bytes in a file, since the grammar demands them rather than choosing them freely. Boundaries should therefore be pushed away from interiors and from closing positions, and constituent starts inherit boundaries largely by elimination, not because they are points of maximal grammatical choice, but because everywhere else is comparatively more constrained. This motivates P2: alignment should be clearly stronger at constituent *starts* than at their *ends*.

**Two named failure modes.** This asymmetry predicts exactly two ways the mechanism should break, against two different metrics:

- Some constituents open **deterministically**: `if`, `for`, `while` are each followed by a forced token, and `def` by a name. Branching factor here is  $\approx 1$  at the point grammar resets, so BLT should *miss* these starts, costing **recall** (P4).
- High entropy also arises where **no constituent opens at all**: mid-identifier, inside a string literal, inside a comment. These are lexical choice points, not grammatical ones. This costs **precision**, and predicts precision should exceed recall overall.

**A worked example.** Table 1 walks a single line of C++, `if (x > 0) { total += 1; }`, through every case the mechanism distinguishes. The `i` of `if` is high-entropy and a genuine constituent start: correct. The `(` after `if` is low-entropy (grammar forces it): correctly quiet, despite also beginning a sub-expression. The `x` of `x > 0` can be high-entropy purely because identifiers are lexically unpredictable, with no constituent genuinely opening there: the precision cost named above. The closing `;` and `}` are low-entropy and not starts: correctly quiet. That third row, aggregated across a corpus, is predicted to erode precision; its mirror image, a forced opener like `if` itself being missed, is predicted to erode recall, and is exactly the prediction that reverses in §5.

| Position                              | Entropy | Split? | Case                   |
|---------------------------------------|---------|--------|------------------------|
| <code>i</code> of <code>if</code>     | high    | yes    | correct                |
| <code>(</code> after <code>if</code>  | low     | no     | missed, by design      |
| <code>x</code> of <code>x&gt;0</code> | high    | yes    | lexical, not syntactic |
| <code>;</code> <code>}</code>         | low     | no     | correct                |

Table 1: The mechanism applied to `if (x > 0) { total += 1; }`.

**Prose as the contrast case.** Across a natural-language sentence, branching factor is comparatively flat: content words are hard to predict everywhere, function words easy everywhere, and no rule forces a clause to end with a specific byte the way `;` or `}` does in code. The mechanism therefore predicts *weaker* structural anchoring in prose than in code (P3). Prose is a built-in negative control: if code and prose fail to separate, the experiment is measuring  $\tau$ , not grammar.

---

```

corpus bytes (16,300 files)
  → Stream A: BLT patcher
    (itazap/blt-1b-hf, 512-byte
    sliding window restored,  $\tau = 1.3354$ 
    nats) → per-byte entropy +
    patch boundaries
  → Stream B: tree-sitter ASTs
    (code), benepar constituents
    (prose), whitespace/word
    baselines, O4 noised variants
      ↓
Stream C (scoring): byte-exact
contract, one-to-one matching,
20/80 calibration/test split,  $k$ 
frozen on calibration only,
fixed-EOF null (10,000 perms),
file bootstrap

```

---

Figure 1: The pipeline end to end. Stream A’s boundary generation and Stream B’s ground-truth extraction are independent and only meet at Stream C’s scoring stage.

## 4 Experimental Design

Figure 1 summarizes the full pipeline end to end: BLT’s own entropy and boundaries on one side, independently derived AST/constituency ground truth on the other, meeting only at the scoring stage, so that the ground-truth extraction can never leak into, or influence, the boundary-generation step.

### 4.1 Null model and baselines

Any boundary set with approximately the right density scores some nonzero overlap with AST node starts purely by chance, since mean patch length is itself roughly pinned by  $\tau$ . A raw alignment percentage is therefore close to uninformative on its own, and two separate comparisons are required before any number can be interpreted.

**$H_0$  (density-matched null).** For each file, BLT’s own patch lengths are shuffled into a new random order and laid end-to-end across the same file (same boundary count, same length distribution, randomly placed), scored against the same AST. This is repeated as a 10,000-resample permutation test; only the margin of BLT’s real score *over*  $H_0$  is treated as evidence.

**Structure-blind baselines.**  $H_0$  is blind to structure by construction, but does not rule out a second, code-specific confound: most real AST node starts already sit immediately after a newline and indentation change. We measured this directly on our own corpus: 31.8% of Python AST node starts and 22.5% of C++ ones sit exactly at a newline-plus-indent-change position. A trivial rule, a boundary

after every newline, at every indentation change, or at every word start (for prose), would therefore already score respectably with zero grammatical understanding. P1 is only informative if BLT beats this baseline too, not merely random chance, which matters most in Python, where indentation *is* the grammar.

### 4.2 Corpus, splits, and scoring protocol

The corpus totals 16,300 files: Python (CodeSearchNet), C++ (The Stack, via the bigcode/the-stack-smol sample), and English prose (WikiText-103), tree-sitter parsed for code and benepar-parsed for prose, with every byte offset validated against the source text before being trusted. Each language is split deterministically into a 20% calibration set and an 80% held-out test set (test split: 6,400 Python, 6,400 C++, 240 English files). The alignment tolerance  $k$  is frozen per language on the calibration split alone, before any test-split number is generated. In this run,  $k = 0$  in all three languages, meaning an exact byte-offset match was already sufficient and no slack needed to be granted.

Scoring uses maximum-cardinality one-to-one matching between predicted boundaries and target constituent-start offsets within  $k$  bytes, so a single prediction cannot be credited against multiple nearby targets. With  $T$  matched pairs,  $P$  total predictions, and  $G$  total targets, pooled across all files in a language,

$$\text{precision} = \frac{T}{P}, \quad \text{recall} = \frac{T}{G}, \quad \text{F1} = \frac{2T}{P + G}. \quad (2)$$

Length-capped boundaries (a patch that hit BLT’s maximum-length cap before entropy crossed  $\tau$ ) are excluded from  $P$ , since they reflect a budget constraint, not an entropy decision, and have nothing to do with grammar. The density-matched null uses a fixed-EOF permutation: a file’s (length, termination-reason) pairs are shuffled for every patch except the last, which stays in place so the file’s final byte is never itself a scored boundary. This preserves file length, the patch-length multiset, and the total boundary count exactly. 10,000 such permutations are drawn per file for the test-split scoring; confidence intervals come from a separate, paired per-file bootstrap (2,000 replicates), so the same file’s clean and noised (O4) scores, or its start and end scores, are always resampled together rather than independently.

### 4.3 Checkpoint fidelity

The only public checkpoint for this model, `itazap/blt-1b-hf`, is a third-party conversion, so its behavior cannot be assumed to match the architecture in the source paper. Before any entropy or boundary number was produced, we verified coherent generation, sensible boundaries on hand-constructed inputs, and, most importantly for §3, that the effective context window matches the 512-byte sliding window the original work reports, since a silently absent window would change what "preceding context" means in every claim here. The Hugging Face port was found to omit this restriction by default, applying full, unbounded attention instead; the window was restored before any reported number was generated, and this gate is re-run for every extraction batch rather than checked once. Table 2 summarizes the full gate battery; two gates were amended after the first run because their declared criteria were mis-specified (an unneeded EOS-token expectation and an uncalibrated random-byte threshold), not because of a pipeline defect, and both amended versions pass.

| Gate | What it checks                                          | Result |
|------|---------------------------------------------------------|--------|
| G1   | weights: key match, bf16→fp32 upcast, parameter count   | pass   |
| G2   | tokenizer: BOS + byte offsets (EOS criterion amended)   | pass   |
| G3   | math: HF port vs. independent reference, max logit gap  | pass   |
| G4   | entropy: Shannon entropy in nats, float64 cross-check   | pass   |
| G5   | boundaries: HF's own patch rule, 28/28 battery inputs   | pass   |
| G6   | context: zero influence beyond the 512-token window     | pass   |
| G7   | behaviour: low entropy on repeats, high on random bytes | pass   |
| G8   | determinism: identical reruns are bit-identical         | pass   |

Table 2: Checkpoint-fidelity gate battery (full detail in `docs/STREAM_A_FIDELITY.md`).

### 4.4 Robustness to code-level noise

A structural signal that is actually tracking grammar should degrade gracefully under perturbations that preserve logical validity but disturb surface form, and collapse if it is purely a surface-pattern artifact. We construct a noised variant of the code corpus using four noise types applied independently: missing semicolons, localized indentation changes, identifier-naming inversions (`camelCase` ↔ `snake_case`), and typos in-

jected into syntax keywords. Noised files are re-parsed with tree-sitter, and any file whose mutation produces a tree that fails to parse cleanly is discarded from the noised evaluation, so that degraded alignment under noise cannot be attributed to degraded ground truth rather than to the noise itself. We report the change in the primary start-alignment margin, and in mean bytes-per-patch, between the clean and noised versions of the same files, for the noise type producing the largest drop (indentation changes, for both code languages).

## 5 Results

All numbers below are generated directly from the frozen, checksummed run `results/final/c-v1-2721954/`.

Table 3 shows BLT clears the null decisively in every language. The margin is 4 to 5 times the null's own score, with confidence intervals nowhere near zero. But it is beaten by the simplest structure-blind rule available in every language, by a wider margin than it beat the null in the first place. Precision is also far below recall, the opposite of the corollary predicted in §3 (§6 returns to this).

Table 4 shows P2 and O4 land exactly as predicted: boundaries are reliably weaker at constituent ends than starts, and the structural signal survives code-level noise (typos, indentation shifts, naming-convention swaps) with only a small, graceful degradation. C++ also carries the largest within-file bytes-per-patch variance of the three languages: a single C++ file typically mixes long, cheap, boilerplate stretches (headers, declarations, brace blocks) with short, dense logic, so the patcher alternates between long and short patches within one file in a way that comparatively homogeneous prose paragraphs do not. P4 reverses outright: forced openers (`if/for/while/def`) align *more* strongly than open-ended constructs, not less. P3's cross-domain reversal is visible by comparing English's margin in Table 3 (+0.348) against Python's and C++'s (+0.120, +0.129). Prose shows the *strongest* alignment of the three languages, the opposite of the prediction that code's rigid grammar should anchor structure more tightly than prose.

Figure 2 shows the start-alignment margin broken down by AST depth, the granularity at which "alignment" is well defined in the first place, since every byte sits inside several nested constituents simultaneously.

|                               | Python                  | C++                     | English                 |
|-------------------------------|-------------------------|-------------------------|-------------------------|
| BLT F1 vs. null               | 0.152 vs. 0.032         | 0.165 vs. 0.036         | 0.495 vs. 0.147         |
| Margin over null [95% CI]     | +0.120 [0.117, 0.122]   | +0.129 [0.125, 0.134]   | +0.348 [0.341, 0.354]   |
| Precision / recall            | 0.083 / 0.822           | 0.095 / 0.646           | 0.342 / 0.894           |
| Strongest baseline (type, F1) | indentation, 0.394      | word-start, 0.339       | word-start, 0.769       |
| BLT – best baseline [95% CI]  | −0.242 [−0.247, −0.237] | −0.173 [−0.181, −0.167] | −0.274 [−0.280, −0.268] |

Table 3: P1: start-alignment F1, BLT vs. density-matched null, vs. best structure-blind baseline.

|                               | Py     | C++    | En     |
|-------------------------------|--------|--------|--------|
| P2: end-boundary margin       | −0.017 | −0.035 | −0.079 |
| P3: mean bytes/patch          | 5.68   | 7.86   | 3.77   |
| P4: recall margin, forced     | +0.759 | +0.691 | n/a    |
| P4: recall margin, open-ended | +0.611 | +0.451 | n/a    |
| O4: margin, clean             | +0.115 | +0.131 | n/a    |
| O4: margin, worst noise       | +0.099 | +0.118 | n/a    |

Table 4: Secondary predictions. P2 and O4 confirmed; P4 reversed (forced openers align *more* strongly, not less). P3’s reversal is visible by comparing English’s start-margin in Table 3 (+0.348) against Python’s and C++’s (+0.120, +0.129).

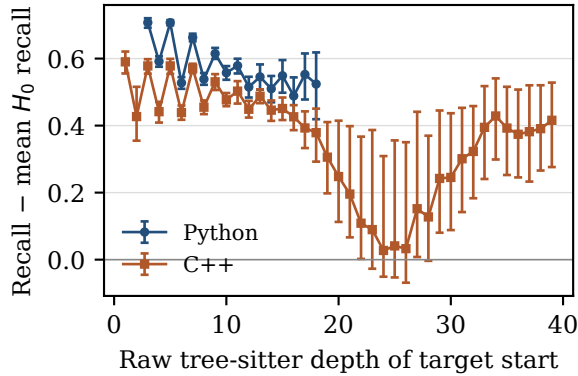


Figure 2: Start-alignment margin over the null, by AST depth.

## 6 Discussion

Two of four secondary predictions reversed outright, and the primary prediction itself (P1) is beaten by a baseline that understands nothing about grammar. Reading these as three independent failures would miss the actual finding: all three point to the same underlying cause.

**Why forced openers align better, not worse (P4).** The original mechanism argument concerned branching *inside* an already-committed construct: once three tokens into `if (x > 0)`, little is left to decide. But at the very first byte of a new line, the model does not yet know whether an `if`, a `for`, a bare assignment, or a return statement is about to begin, which on its own would

predict no difference between forced-opener and open-ended constructs, not the stronger-than-open-ended result observed. The better-supported explanation follows directly from Table 3: since most of BLT’s signal already tracks line and word boundaries rather than grammar, the **most consistently formatted** constructs, `if/for/while/def`, near-universally written on a fresh line with standardized indentation, should show the strongest alignment for that reason: formatting consistency, not branching factor.

**Why English aligns more strongly than code (P3).** English’s own strongest baseline in Table 3 is word-start splitting, at F1 = 0.769, far higher than either code language’s best baseline. Nearly every phrase-level constituent in a benepar parse (NP, VP, PP) begins exactly at a word start, and word starts are themselves strong, frequent entropy spikes given English’s much larger vocabulary than source code’s repetitive, boilerplate-heavy one. BLT’s entropy spikes and benepar’s constituent starts plausibly both key off the same regularity, the start of a new word, without requiring any syntactic sensitivity beyond that.

**One finding, three appearances.** Read together, P1’s baseline result, P4’s reversal, and P3’s reversal are most parsimoniously explained as the same finding appearing three times: **BLT’s entropy-triggered boundaries are substantially explained by surface segmentation (where lines and words begin), rather than by grammatical branching specifically.** This is a coherent, falsifiable conclusion, not an inconclusive null, and matches what the mechanism (§3) predicted could go wrong: lexical and surface-level entropy swamping the grammatical signal was named in advance as the sharpest failure mode, and the precision/recall asymmetry (precision far below recall, not above it) is independent evidence pointing the same direction.

**Why this result was not obvious in advance.** The mechanism in §3 was built to be diagnos-

able regardless of outcome. Uniformly high alignment everywhere would have indicated an under-trained patcher blind to all structure; uniformly low alignment would have indicated density-limited boundaries with no structure-driven placement at all. What we observe is neither: a real margin over the null, combined with a specific, reproducible pattern of losing to baselines and reversing at exactly the two points the mechanism named in advance as vulnerable. That the predicted failure mode is the one that occurs is itself evidence for the mechanism’s correctness as an account of *how* the patcher behaves, even though the headline hypothesis it was built to test is not supported.

We hold this interpretation with appropriate caution: it is our reasoned account of *why* the reversals occurred, argued from the data and from the mechanism in §3, not a separately, independently verified causal claim. Ruling out the surface-segmentation account directly, for instance by conditioning P4’s stratification on formatting consistency directly rather than inferring it, is identified as concrete future work in §7.

## 7 Remaining Work

Two objectives from the original proposal remain open, both explicitly scheduled for after the structural-alignment results reported here:

**Memory-unsafe compute vs. identifier-entropy confound (O5).** Does any extra compute BLT allocates to memory-unsafe C++ operations (`new/delete/*/&`) survive conditioning on nearby identifier-token entropy, or is it fully explained by that surface confound? Region sampling, unsafe-operation masks, and identifier spans are already computed and require review before any claim is drawn from them.

**Cross-language structural recognition (O3).** A direct, depth-normalized comparison of how Python’s indentation-based grammar and C++’s bracket-based grammar are each reflected in patch placement, building on the raw AST-depth column already produced for every file (Figure 2).

Beyond these, §6 suggests a direct, falsifiable follow-up: if formatting consistency (not branching factor) explains P4’s reversal, that predicts a testable correlate. Constructs with higher measured formatting consistency (e.g. lower variance in the column/indentation of their opening byte across the corpus) should show higher alignment margins, independent of whether they are a deterministic

or open-ended type. This would let the surface-segmentation account be tested directly rather than argued from pattern alone, and is the most direct next step before any claim here is submitted for external review.

### 7.1 Timeline

Table 5 gives a concrete phasing for the work in this section.

| When     | What                                                                                                                                                                      |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Oct 2026 | O5 confound analysis: review computed region/mask/identifier-span data, condition on identifier entropy, draw a conclusion.                                               |
| Nov 2026 | O3 cross-language depth normalization; run the formatting-consistency follow-up test proposed above. Complete write-up for your project structured like a research paper. |

Table 5: Planned timeline for the remaining work in §7.

### 7.2 Resource Availability

Code, the frozen corpus manifest, and all results referenced in this report are available in the project’s GitHub repository.<sup>1</sup> The BLT checkpoint used is the public, pre-existing conversion `itazap/blt-1b-hf`.<sup>2</sup> No model was trained or fine-tuned for this project, so no Weights & Biases project or link applies.

## 8 Conclusion

An unsupervised, compression-only signal, BLT’s entropy-triggered patch boundaries, does align with syntactic constituent starts far beyond chance, in both code and prose. But not for reasons specific to grammar: it is beaten by rules that understand nothing beyond where a line or word begins, and its strongest alignment appears in prose, not the grammatically rigid code languages the hypothesis predicted would anchor it most tightly. The evidence is most consistent with BLT’s byte-level entropy tracking surface segmentation rather than grammatical branching, a specific conclusion obtained precisely because the experiment was designed so a null result would be as interpretable as a positive one.

<sup>1</sup>[github.com/dawnbliss-coder/ESCAPE](https://github.com/dawnbliss-coder/ESCAPE) (truncated for width; link target is the complete URL).

<sup>2</sup><https://huggingface.co/itazap/blt-1b-hf>

## Limitations

The BLT checkpoint used (itazap/blt-lb-hf) is a third-party conversion, not the original authors’ release; we verified its 512-byte sliding-window behavior rather than assuming it, since that window length is load-bearing for the mechanism argued in §3. English constituency targets come from benepar’s *predicted* parses, not hand-annotated treebank gold. About 28% of C++ files trigger tree-sitter’s error-recovery; these files are kept and flagged rather than excluded, since exclusion would bias the C++ sample toward simpler, less representative files. CodeSearchNet’s Python samples are individual functions that may share a source repository, so per-file bootstrap intervals need not generalize across independent codebases. Objective 4’s noise injection uses one noise rate per noise type rather than a sweep across rates. The tolerance  $k = 0$  selected by calibration in all three languages means this report’s alignment numbers require an exact byte-offset match; a nonzero  $k$  would mechanically raise every reported F1 by crediting near-misses, so the numbers in Table 3 should not be compared against a different tolerance protocol without accounting for that. Finally, the structure-blind baselines (newline, indentation-change, word-start) are deterministic rules with no density-matching of their own; BLT is compared against their raw F1 directly, the more conservative comparison, but this means the baseline numbers in Table 3 are not on identical statistical footing to the null-model margin reported alongside them.

## References

- Michael R. Brent. 1999. [An efficient, probabilistically sound algorithm for segmentation and word discovery](#). *Machine Learning*, 34(1-3):71–105.
- Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. 2022. [CANINE: Pre-training an efficient tokenization-free encoder for language representation](#). *Transactions of the Association for Computational Linguistics*, 10:73–91.
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. [CodeBERT: A pre-trained model for programming and natural languages](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547. Association for Computational Linguistics.
- Sharon Goldwater, Thomas L. Griffiths, and Mark Johnson. 2009. [A Bayesian framework for word segmentation: Exploring the effects of context](#). *Cognition*, 112(1):21–54.
- John Hewitt and Christopher D. Manning. 2019. [A structural probe for finding syntax in word representations](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4129–4138. Association for Computational Linguistics.
- Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. [CodeSearchNet challenge: Evaluating the state of semantic code search](#). *arXiv preprint arXiv:1909.09436*.
- Nikita Kitaev and Dan Klein. 2018. [Constituency parsing with a self-attentive encoder](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2676–2686, Melbourne, Australia. Association for Computational Linguistics.
- Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. 2022. [The stack: 3 TB of permissively licensed source code](#). *arXiv preprint arXiv:2211.15533*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. [Pointer sentinel mixture models](#). In *International Conference on Learning Representations*.
- Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srini Iyer. 2025. [Byte latent transformer: Patches scale better than tokens](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258, Vienna, Austria. Association for Computational Linguistics.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. [Neural machine translation of rare words with subword units](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725. Association for Computational Linguistics.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. [ByT5: Towards a token-free future with pre-trained byte-to-byte models](#). *Transactions of the Association for Computational Linguistics*, 10:291–306.